Mohammed Mohi Uddin

Hyderabad, India | t.d.m.mohi@gmail.com | +91 78424 49809 | LinkedIn | GitHub | Portfolio

AI systems and backend engineer

I build systems that have to survive a real constraint, and I write down which numbers were measured and which were assumed. Fourteen projects across AI systems, computer vision, embedded hardware and developer tooling. One launched product on live payment infrastructure, and four projects paused or retired with the reason recorded.

Experience

Founder and sole engineer LAUNCHED JUN 2026 — Feb 2026 – present

DebugAI · a debugging system for developers and AI agents, debugai.io

- Designed and shipped every layer: the editor extension, an Express API gateway, a FastAPI inference engine, a Next.js dashboard and marketing site, PostgreSQL with pgvector, Redis, and the deployment pipeline across Vercel and Railway.
- Three distribution channels rather than one: the extension on the VS Code Marketplace and on Open VSX, which covers Cursor, Windsurf and VSCodium, and `@debugai/mcp` published to npm as a standalone MCP server so a coding agent calls the same engine with no editor involved. The MCP server resolves the files a stack trace names under a repository root allowlist with a containment check after realpath.
- 23,000+ combined downloads across VS Code, Open VSX and npm and 430+ signed-up developers as of November 2026, with no paid marketing. Over half of all debugs now come from coding agents calling the MCP server on their own.
- Context assembly: parse the stack trace, force inclusion of the files it names, then retrieve related code the trace never mentions using pgvector embeddings over the repository, inside a managed token budget. A second retrieval pass runs when the model reports missing context, under a 40 second deadline with a fallback to the first pass result.
- Cost control: a SHA-256 cache key over error text, project and framework hint checked in Redis before any model call, plus difficulty based routing between a small and a large model and a cross provider fallback that preserves the response shape on an upstream 429 or 503.
- Error memory: a SHA-256 signature per error, recurrence tracked per project, and a fix confirmed by a human promoted to lead the next answer for that signature. Project scoped only, so a fix never leaks across users.
- Billing on production payment keys: raw body HMAC verification before the JSON parser, two phase idempotency keyed on the provider event id, and an effective tier guard evaluated in the request path so an expired trial is enforced even when the hourly cron fails. That cron did fail once, pointed at the wrong host, and returned 404 for days.
- Security posture: session identifiers in Redis rather than the secret in a cookie, constant time secret comparison, a CSRF gate bound to the same middleware that grants admin authority, and a fail closed policy when the session store is unavailable.
- 879 commits between February and August 2026. 519 engine tests, 32 API test suites, 44 written session logs, and continuous integration on every push.

`TypeScript, Python, Next.js 15, Express, FastAPI, PostgreSQL, pgvector, Redis, Docker, nginx, Railway, Vercel, Paddle`

Summer Training Intern, Generative AI with Machine Learning — Jun – Jul 2025

RemarkSkill Education with IIT Hyderabad

- Built a retrieval augmented question answering system over a document corpus: embedding based retrieval by cosine similarity, document chunking, context injection, generative response.
- Found that chunk boundaries, not prompt wording, decided answer quality. Most of the tuning went into segmentation rather than generation.
- Recognised at programme close for technical soundness and collaboration.

`Python, vector databases, embeddings, RAG`

Engineering projects

MatchLedger AI · reconciliation at 100,000 rows a side — May 2025 – present

Founder, sole engineer. Paused pending company registration.

- Three passes ordered cheapest first. A vectorised hash join on amount, date and reference. A tolerance pass on exact amount with a configurable date window and greedy closest date assignment. A fuzzy narration pass run only on the remainder, blocked by amount bucket and date window so it never becomes an O(n×m) scan.
- Benchmarked at 100,000 against 98,000 rows in 4.25 seconds, 95% match rate, zero API calls, zero mismatches against synthetic ground truth with a known answer key.
- Replaced an earlier embedding based engine that called an API per unresolved row. Both engines remain in the repository, which makes the comparison reproducible.
- Every match records the pass and the basis that produced it, because a match without an audit trail is useless to whoever signs off on it. Results stored encrypted at rest.
- Celery and Redis for background jobs, four service Docker Compose, JWT and Google OAuth, React 19 frontend.

`Python, FastAPI, pandas, PostgreSQL, Celery, Redis, React, Docker, AWS EC2`

Inkline · handwriting to font, no server involved — Aug 2026

- Template geometry, scan, trace and build as four independently testable pure modules. Corner markers plus a rotation pip define a reference frame, so after homography every cell is at a known position and no letter level alignment guessing is needed.
- Tracer written from scratch: Moore neighbour boundary following with hole detection, staircase smoothing, corner detection over an angular window, Schneider least squares cubic fitting with Newton-Raphson reparameterisation, winding normalised so counters render as holes.
- opentype.js cannot write kerning, so the kern table is assembled by hand and spliced into the binary with the table directory rebuilt, offsets re-paged and the head checksum recomputed.
- End to end proof: a synthetic tilted, shadowed, noisy photograph drives a headless Chromium through the studio, the produced OTF is downloaded and parsed back.

`TypeScript, Next.js, Canvas, opentype.js, pako, IndexedDB, Playwright`

SS Categorise · multimodal classification — 2024, rebuilt Feb – Apr 2026

- Version one: a fine tuned ResNet on roughly 12,000 scraped images with K-means proposing groupings before labelling. It plateaued and adding data did not help.
- Version two: EfficientNet-B3 image embedding concatenated with a 500 term TF-IDF vector over OCR text, classified by logistic regression. 96.92% on a 260 image held-out test set across 20 classes.
- Kept the classification head linear on purpose so the contribution of each half of the fused vector stayed inspectable. The two worst classes both fail for the same reason: no readable text, so the fusion degrades to vision alone.

`PyTorch, EfficientNet-B3, OCR, TF-IDF, scikit-learn, Streamlit`

Deconstruct · video breakdown studio — Jul 2026

- Separates audio stems with Demucs, detects every cut, extracts per shot camera motion with optical flow and matches it to a named CSS cubic-bezier, reads caption timing with OCR, and reports the colour grade.
- Marks each cut green or red depending on whether it lands on a musical beat, which is the single comparison that teaches edit rhythm.
- Analysis streams progress over server sent events per stage, queues concurrent requests rather than running them in parallel, and skips stem separation above twelve minutes of runtime so the rest still completes.

`FastAPI, ffmpeg, PySceneDetect, librosa, Demucs, OpenCV, RapidOCR, React, Vite`

SecAudit · static security scanner, zero dependencies Mar 2026

- One Python file, standard library only. 60+ rules across the OWASP Top 10 covering secrets, injection, authentication flaws, deserialisation, missing rate limits and misconfiguration, for Python, JavaScript, TypeScript, React, FastAPI, Node, Go, Ruby and PHP.
- Text, HTML or JSON output with a severity filter so it can gate a CI pipeline. Written after doing a full manual remediation pass on my own codebase, so most rules are mistakes I had made.

`Python standard library`

Telecare Nabha · rural telemedicine, Smart India Hackathon 2025 2025

- Built for a district where 31% of households have internet access and the civil hospital runs 11 doctors against 23 sanctioned posts for 173 villages.
- Dual channel by design: a smartphone app and an IVR path over a button phone, offline first health records that reconcile when a connection appears, and live pharmacy stock so a patient does not lose a day of wages travelling to an empty shelf.

`Flask, React, Vite, SQLite, IVR`

Hardware and research work

OptiXon · health sensing eyewear, full hardware and firmware design 2024 – Jul 2026

Founder and technical lead. Paper phase complete, no prototype built.

- PPG, six axis IMU and skin temperature sensing on a 138 × 10.5mm four layer flex-rigid board inside an eyeglass temple, with BLE and hardware battery protection.
- Board, enclosure and routing generated by Python against KiCad and FreeCAD rather than drawn, so a change is re-run and re-verified rather than redrawn. Electrical rule check clean, netlist machine verified, fabrication pack exported, firmware builds against the nRF5 SDK.
- 51 findings logged as symptom, root cause and fix. Two are field failure class safety bugs found before fabrication, including a battery protection pin mapping error that would have left a lithium cell unprotected while behaving normally in every functional test.
- Projected 18 to 24 hour battery life and sub 28°C skin contact temperature. Both are design calculations and are labelled as such, because nothing has been built.

`nRF52832, MAX30102, LSM6DSOX, KiCad, FreeCAD, ngspice, nRF5 SDK, C`

BeyondVision · spectral imaging specification 2026

- A ten document design set for spectral capture on phones and cheap add on hardware, built around a provenance rule: every pixel is tagged MEASURED, RECONSTRUCTED or SIMULATED and nothing exports without the tag.
- Establishes what is physically impossible before designing anything. Silicon stops at 1107nm because its bandgap is 1.12 electronvolts, so thermal imaging from a phone camera cannot exist, and five hardware tiers from zero to roughly 250 dollars are specified against that boundary.

`Optics, RAW capture, spectral reconstruction, calibration procedure`

Skills

Languages: Python, TypeScript, JavaScript, C, SQL

Backend and data: FastAPI, Express, Node.js, Celery, Redis, PostgreSQL, pgvector, SQLAlchemy, pandas, REST and webhook design

Frontend: React, Next.js App Router, Vite, Canvas, server sent events, design systems

Machine learning: PyTorch, TensorFlow, scikit-learn, OpenCV, EfficientNet and ResNet fine tuning, embeddings and retrieval, model routing, evaluation and error analysis

Infrastructure: Docker and Compose, AWS EC2, Railway, Vercel, nginx, GitHub Actions, Sentry, migration discipline

Hardware: KiCad, FreeCAD, ngspice, nRF52 and ESP32, MEMS sensors, PPG and IMU signal handling, power budgeting

Security: OWASP Top 10, HMAC webhook verification, constant time comparison, CSRF and session design, threat modelling

Education

B.Tech, Computer Science Engineering (Data Science) Aug 2023 – Aug 2027

Swami Vivekananda Institute of Technology, Hyderabad

- Coursework: machine learning, computer vision, natural language processing, data structures and algorithms, database systems, operating systems.

Recognition and validation

- Second place, research paper competition, Techno Udbhav 2025 inter college technical festival, for OptiXon health sensing eyewear.
- Smart India Hackathon 2025, team Glitch Guardians, problem statement PS25018: telemedicine access for rural healthcare in Nabha, Punjab.
- OptiXon design and workflow reviewed across multiple sessions with M.Tech students at IIT Hyderabad and a working hardware professional, who confirmed feasibility and suggested changes. Every design, simulation and firmware decision is my own.
- Certificates: Generative AI with Machine Learning, Ethical Hacking, and Data Science workshops, all IIT Hyderabad.